

Unit 3 Assignment – Student Work Summary

Name: Pragma Agarwal
Course: Btech CSE Core
Section:A
Roll No.:2501010161
Subject: Data Structures

Title: Analysis of Sorting Algorithms (Insertion, Merge, Quick)

1. Objective

To implement Insertion Sort, Merge Sort, and Quick Sort algorithms and analyze their performance on different types of datasets (random, sorted, and reverse) for varying input sizes.

2. Tools Used

- Programming Language: Python
 - Software: VS Code
-

3. Methodology

- Implemented three sorting algorithms:
 - Insertion Sort
 - Merge Sort
 - Quick Sort
 - Generated datasets of sizes:
 - 1000, 5000, 10000
 - For each size, created three types of input:
 - Random data
 - Already sorted data
 - Reverse sorted data
 - Measured execution time using `time.perf_counter()`
 - Recorded results in a timing table
-

Results (Timing Table)

Timing Table (seconds)				
Size	Type	Insertion	Merge	Quick
1000	random	0.082858	0.003077	0.002529
1000	sorted	0.00037	0.003471	0.003635
1000	reverse	0.112006	0.003461	0.003373
5000	random	1.529462	0.026201	0.015197
5000	sorted	0.001707	0.017761	0.016399
5000	reverse	2.986227	0.021459	0.014784
10000	random	6.22641	0.092826	0.04605
10000	sorted	0.003744	0.034666	0.038709
10000	reverse	14.862634	0.043286	0.043522

PS C:\Users\kcs\OneDrive\Desktop\data structure>

Theory of Sorting Algorithms

1. Insertion Sort

Insertion Sort is a simple comparison-based sorting algorithm that builds the final sorted array one element at a time. It works by taking each element and inserting it into its correct position in the already sorted part of the array.

The algorithm starts from the second element and compares it with the previous elements, shifting larger elements to the right until the correct position for the current element is found.

- **Time Complexity:**
 - Best Case: $O(n)$ (when the array is already sorted)
 - Average Case: $O(n^2)$
 - Worst Case: $O(n^2)$ (when the array is reverse sorted)
- **Space Complexity:** $O(1)$ (in-place)
- **Stability:** Stable (maintains relative order of equal elements)
- **Key Feature:** Efficient for small or nearly sorted datasets

2. Merge Sort

Merge Sort is a divide-and-conquer algorithm that divides the array into two halves, recursively sorts each half, and then merges the sorted halves to produce the final sorted array.

The merging process compares elements from both halves and arranges them in sorted order. This continues until the entire array is sorted.

- **Time Complexity:**
 - Best Case: $O(n \log n)$
 - Average Case: $O(n \log n)$

- Worst Case: $O(n \log n)$
 - **Space Complexity:** $O(n)$ (requires extra memory for merging)
 - **Stability:** Stable
 - **Key Feature:** Consistent performance regardless of input type
-

3. Quick Sort

Quick Sort is also a divide-and-conquer algorithm. It works by selecting a pivot element and partitioning the array into two parts:

- Elements smaller than the pivot
- Elements greater than the pivot

After partitioning, the same process is recursively applied to both subarrays.

- **Time Complexity:**
 - Best Case: $O(n \log n)$
 - Average Case: $O(n \log n)$
 - Worst Case: $O(n^2)$ (when partitions are highly unbalanced)
 - **Space Complexity:** $O(\log n)$ (due to recursion stack)
 - **Stability:** Not stable
 - **Key Feature:** Very fast in practice but performance depends on pivot selection
-

Quick Sort Worst Case Explanation

The worst case occurs when the pivot divides the array into highly unequal parts. For example, if the array is already sorted and the last element is chosen as the pivot, one partition becomes empty while the other contains all elements. This leads to $O(n^2)$ time complexity.

5. Observations

- Insertion Sort performs very fast on sorted or nearly sorted data but is slow for reverse order due to many shifts.
 - Merge Sort shows consistent performance for all input types because its time complexity is always $O(n \log n)$.
 - Quick Sort performs very efficiently on average but becomes slow when the pivot selection is poor.
-

7. Stability and In-Place Comparison

Algorithm	Stable	In-Place
Insertion Sort	Yes	Yes
Merge Sort	Yes	No
Quick Sort	No	Yes

8. Conclusion

Insertion Sort is suitable for small or nearly sorted datasets. Merge Sort is efficient and stable but requires extra memory. Quick Sort is generally the fastest in practice but its performance depends on pivot selection and may degrade in the worst case.

9. Learning Outcome

This experiment helped in understanding the practical performance of sorting algorithms, the impact of input type, and concepts like stability, in-place sorting, and time complexity.
